

Prerequisites for the Semantic Embedding Report:

Gaussian Splats, CLIP, DINOv2, SAM, the Lift and FMGS — what each does and how

Suhan — Team Radiance, Department of Computer Science and Engineering,
University of Moratuwa

8 October 2026

Abstract

This companion note explains, at an abstract level, the building blocks used by our semantic layer: the 3D Gaussian Splat it is attached to, the three 2D foundation models that supply features (CLIP for language, DINOv2 for object structure, SAM for object outlines), the relevancy score that turns a phrase into a per-point score, and the two ways of attaching 2D features to a 3D splat (the training-free *lift* and the trained *FMGS* feature field). It contains no results; the main report (`semantic_embedding_report.tex`) builds on these ideas.

Contents

1	The problem in one picture	1
2	3D Gaussian Splatting	1
3	CLIP: images and text in one space	2
4	Relevancy: turning a phrase into a score per point	3
5	DINOv2: what belongs together	4
6	SAM: object outlines (used by variants B and B2)	4
7	The lift: attaching 2D features to Gaussians without training	4
8	FMGS: a trained feature field	5
9	Glossary	6

1 The problem in one picture

We have a photo-realistic 3D model of a room (a *Gaussian splat*) and the photos it was made from. We want to type “red cup” and get back the 3D position of the red cup, so that a drone course can end in front of it. The splat itself knows only colour and geometry; it has no idea what a cup is. So we borrow understanding from 2D image models, which are trained on hundreds of millions of images, and transfer what they see in each photo onto the 3D points (Figure 1).

2 3D Gaussian Splatting

What it is. A 3D Gaussian splat [1] represents a scene as hundreds of thousands of small, semi-transparent, coloured ellipsoids (“Gaussians”). Each Gaussian i has a centre $\mathbf{x}_i$, an orientation and three axis lengths (its shape), an opacity o_i and a view-dependent colour. Our backroom scene has 532,361 Gaussians.

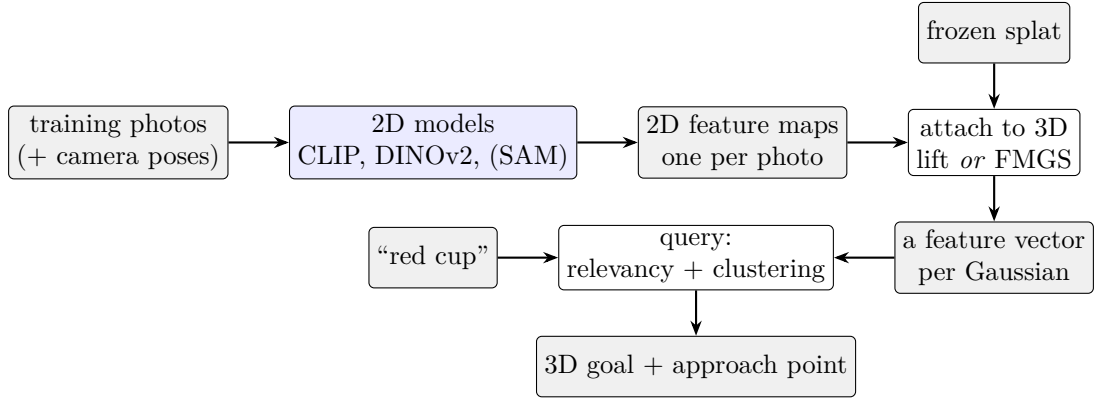


Figure 1: The overall idea: 2D models see the photos, their features are transferred onto the 3D splat, and a phrase is compared against those features.

How an image is rendered. For a camera, every Gaussian is projected to a 2D ellipse on the image. For each pixel, the ellipses covering it are sorted front to back and their colours are *alpha-blended*:

$$C(p) = \sum_i \alpha_i(p) \underbrace{\prod_{j < i} (1 - \alpha_j(p))}_{w_i(p)} c_i, \quad (1)$$

where $\alpha_i(p)$ is the Gaussian’s opacity times its 2D footprint at pixel p , and the product is the light that survives the Gaussians in front. The quantity $w_i(p)$ — the *blend weight* — says how much Gaussian i contributes to pixel p . Two facts matter for us:

1. **Any per-Gaussian vector can be rendered**, not just colour: replace c_i by a feature vector $\mathbf{f}_i$ and the same formula produces a feature image. The rasteriser we use (gsplat 1.0.0) can render up to 512 channels forward, but only 32 channels when gradients are needed, so features are rendered in 32-channel chunks.
2. **Rendering is linear in the per-Gaussian values**, which is what makes the lift (Section 7) possible without any training.

Training a splat. The Gaussians are optimised so that renders match the photos (FiGS uses nerfstudio’s `splatfacto`, 30,000 steps). It also refines each camera’s pose a little. In our work the splat is **frozen**: its Gaussians are never changed again, because drone courses, cached renders and trained policies all depend on them.

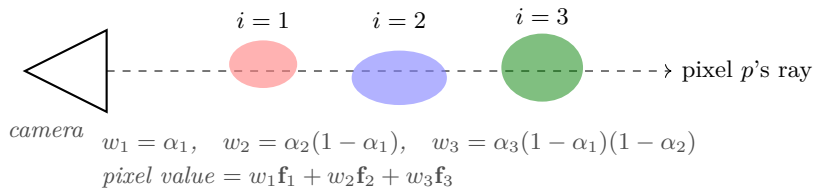


Figure 2: Alpha blending along one pixel’s ray. The blend weight w_i of each Gaussian is how much it contributes to the pixel; front Gaussians hide the ones behind them.

3 CLIP: images and text in one space

What it does. CLIP [2] is a pair of neural networks — an image encoder and a text encoder — trained on hundreds of millions of image–caption pairs so that an image and a caption that describes it map to nearby unit vectors (high cosine similarity), and unrelated pairs to distant ones.

After training, *any* phrase can be compared with *any* image, without a fixed list of classes: this is what makes our queries *open-vocabulary*.

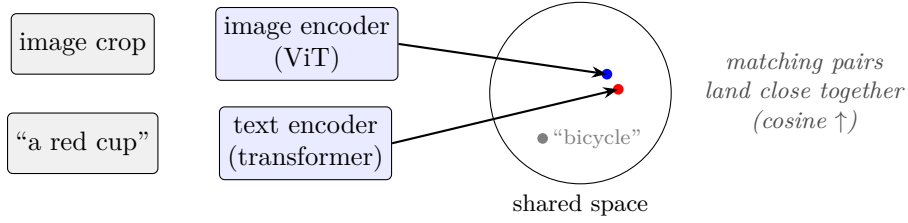


Figure 3: CLIP maps images and text into one space of unit vectors (512-d for ViT-B/16, 768-d for ViT-L/14).

How. The image encoder is a Vision Transformer (ViT): it cuts a 224×224 image into patches (16×16 pixels for ViT-B/16, 14×14 for ViT-L/14), processes them with self-attention, and summarises the whole image into *one* vector. Training uses a contrastive loss: in a batch of N image–caption pairs, each image must be most similar to its own caption among the N . We use OpenCLIP models trained on LAION-2B: ViT-B/16 (512-d) by default and ViT-L/14 (768-d, about $3.5\times$ the compute) in variant C.

Why it is coarse. CLIP describes a *whole image* with one vector; it does not say where in the image things are. To get a per-pixel map we must embed many crops and assign each crop’s vector to the pixels it covers. That is the **crop pyramid**:

- square crops of several sizes (we use 7 scales, from 5 % to 50 % of the image’s short side) slide over the photo with half-crop overlap;
- each crop is resized to 224×224 and embedded;
- every crop’s vector is written at its centre, giving one coarse grid per scale, and the grids are resampled to one common grid (about 71×40 cells per photo).

Small crops see small objects clearly but lack context; large crops see context but drown small objects. How the scales are *combined* — averaged (“Old”), kept separate (variant A), or replaced by object-shaped segments (variant B) — is the main subject of the report.

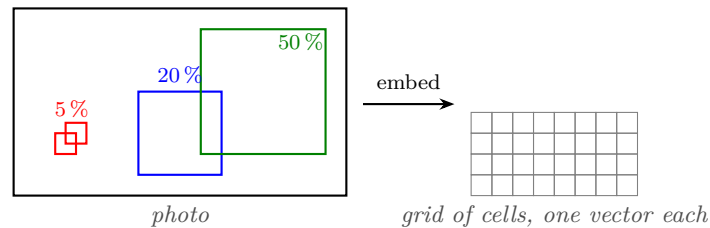


Figure 4: The CLIP crop pyramid: overlapping square crops at several scales are embedded and placed on a grid.

4 Relevancy: turning a phrase into a score per point

A CLIP vector $\mathbf{f}$ at a point and a phrase’s text vector $\mathbf{t}$ give a cosine similarity, but raw similarities are not comparable across phrases. LERF [5] therefore compares the phrase against a few *canonical negatives* (*object*, *things*, *stuff*, *texture*) and asks how much more the point resembles the phrase than the most similar negative:

$$r = \min_j \sigma(T(\cos(\mathbf{f}, \mathbf{t}) - \cos(\mathbf{f}, \mathbf{n}_j))), \quad T = 10. \quad (2)$$

$r = 0.5$ means “no preference”; values near 1 mean the point clearly looks like the phrase. Our query keeps points above a *threshold* relative to the phrase’s own peak (with a fixed lower bound, the *floor*, 0.55), groups them into 3D clusters, and returns the best cluster’s centre. Because r depends on the scale of CLIP’s similarities, the floor is *model-specific*: a different CLIP model needs a different floor (this is the point of variant C2).

5 DINOv2: what belongs together

What it does. DINOv2 [3] is a Vision Transformer trained *without any labels or text*, by self-distillation: a “student” network sees distorted crops of an image and must produce the same features as a slowly-updated “teacher” network that sees other crops, plus a masked-patch prediction task. The result is a feature vector for *every patch* (14×14 pixels) in which patches of the same object or material are similar and object boundaries are sharp.

Why we use it. DINOv2 knows nothing about words, so it cannot answer a query. But it is dense and precise where CLIP is coarse, so it is used as a *helper*: in FMGS it teaches the feature field where objects begin and end (pixel alignment), and in the L-CD query it smooths relevancy within an object and splits candidates that merge two objects. We use ViT-S/14 (384-d), run on the photo resized to multiples of 14 pixels ($\approx 64 \times 36$ patches).

	CLIP	DINOv2
Trained with	image–caption pairs (contrastive)	images only (self-distillation)
Output	one vector per image (crop)	one vector per 14×14 patch
Understands words	yes — open vocabulary	no
Spatial precision	coarse (crop-sized)	sharp (patch-sized, object boundaries)
Our use	answers the query	sharpens/organises CLIP (FMGS loss, L-CD)

Table 1: The two teachers are complementary.

6 SAM: object outlines (used by variants B and B2)

The Segment Anything Model [4] cuts an image into object-shaped masks. Its *automatic mask generator* places a grid of prompt points over the image (32×32 in our setting), predicts masks for each point, and keeps the confident, stable, non-duplicate ones. SAM has no notion of words either; in variant B we feed CLIP each *segment* (its cut-out, background blacked out) instead of square crops, as LangSplat [6] does, so that a CLIP vector describes one object rather than a window mixing an object with its surroundings. We use SAM’s ViT-B model.

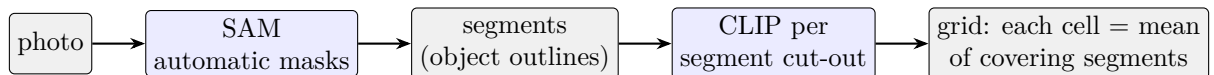


Figure 5: Region-level CLIP: SAM gives the outlines, CLIP describes each region.

7 The lift: attaching 2D features to Gaussians without training

What it does. The lift [9, 10] gives each Gaussian the *weighted average* of the 2D feature-map values at all the pixels it contributes to, in all training photos, weighted by its blend weight:

$$\mathbf{f}_i = \frac{\sum_{\text{photos } v} \sum_{\text{pixels } p} w_i(v, p) F_v(p)}{\sum_v \sum_p w_i(v, p)}. \quad (3)$$

A Gaussian that is clearly visible in a pixel (large w_i) takes much of that pixel’s feature; one hidden behind others (small w_i) takes little. Gaussians never seen by any photo get no features and are marked *unseen*.

How, cheaply. Because rendering is linear in the per-Gaussian values, the numerator is exactly the gradient of $\sum_p \langle \text{render}(\mathbf{f}), F_v \rangle$ with respect to $\mathbf{f}$, and the denominator is the gradient of $\sum_p \text{render}(\mathbf{1})$. So the whole lift is one backward pass of the rasteriser per photo and feature chunk — no optimisation, no hyper-parameters, a few minutes on our GPU.

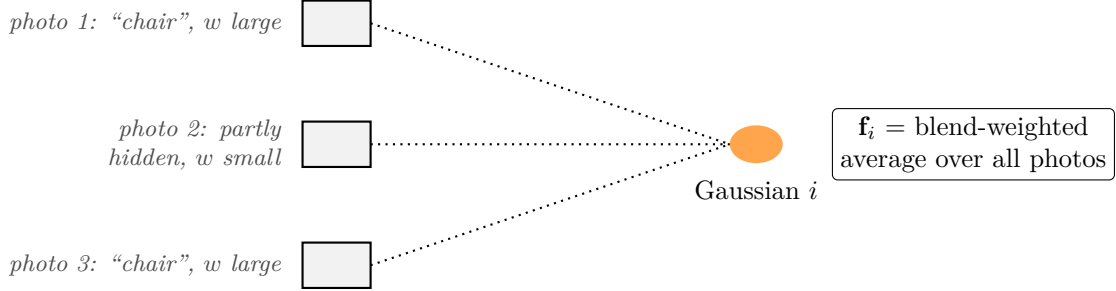


Figure 6: The lift: a Gaussian collects the features of the pixels it is seen in, from every photo.

Strengths and limits. The lift is fast, deterministic and cannot “invent” anything: it reproduces the teacher maps faithfully in 3D (we measured that it loses almost no relevancy). Its quality is therefore bounded by the 2D teacher maps — coarse in, coarse out — and it stores one vector per Gaussian per feature (0.7–2.5 GB per scene).

8 FMGS: a trained feature field

What it does. FMGS [7] (Foundation Model Gaussian Splatting) does not store a free vector per Gaussian. It learns a *function* of 3D position — a neural feature field — that outputs a CLIP vector and a DINO vector at any point. The field is evaluated at the Gaussian centres, the result is rendered like colour, and the rendered maps are compared with the teacher maps of the same photo. Training adjusts only the field (the splat stays frozen).

How. The field has two parts:

- a **multi-resolution hash grid** [8]: 24 grids of increasing resolution (16 to 512 cells per side) cover the scene; each grid’s corners store small learned vectors (8 numbers) in a hash table. A 3D point reads the 8 corners around it in every grid, interpolates, and concatenates the 24 results into a 192-number code. Coarse grids describe large regions; fine grids describe detail.
- small **MLP heads** that turn that code into a 512-d CLIP vector and a 384-d DINO vector.

The loss has three parts: CLIP rendered vs CLIP teacher (Huber), DINO rendered vs DINO teacher (squared error), and *pixel alignment*, which asks neighbouring pixels of the rendered CLIP map to be as similar to each other as DINO says they should be — transferring DINO’s sharp object boundaries to CLIP. After training the field is evaluated once at every Gaussian centre (“baking”) to produce the same per-Gaussian table as the lift.

Strengths and limits. Because nearby points share hash-grid entries, the field *smooths* and can fill in Gaussians seen in few photos, and it can answer arbitrary 3D points, not just Gaussian centres. But it is a trained approximation of the teacher maps: limited capacity, a loss dominated by its largest term, and a training run of 25–45 minutes on our GPU. When the teacher maps contain something misleading (for example zero vectors where SAM found nothing), the field learns it faithfully.

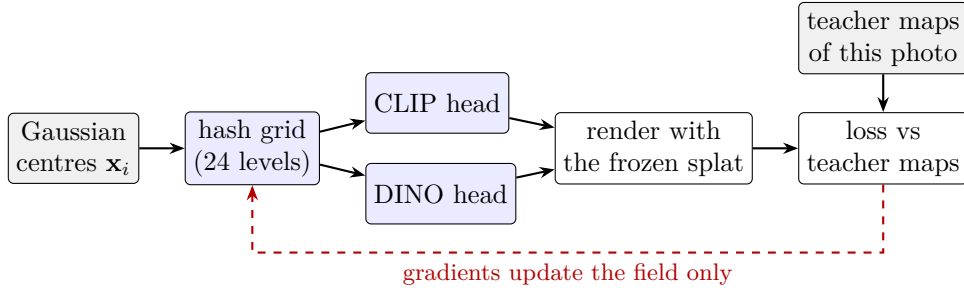


Figure 7: One FMGS training step: the field predicts features at the Gaussians, they are rendered, compared with the photo’s teacher maps, and only the field is updated.

9 Glossary

Term	Meaning
Teacher map	A 2D feature map (CLIP or DINO) computed from one training photo; the “teacher” that the 3D features learn from or are averaged from.
Crop pyramid	CLIP embeddings of square crops at several scales, placed on a grid.
Scale group	Variant A: crops of similar size (small / medium / large) averaged together, kept separate from other groups.
Segment level	Variant B2: SAM segments of similar area (parts / objects / regions), one map each.
Table	The per-Gaussian result: CLIP (and DINO) vector, blend weight, geometry; rows in the same order as Galley’s <code>.splat</code> .
Relevancy	LERF’s per-point score of a phrase versus canonical negatives; 0.5 = indifferent.
Floor	The lowest relevancy that can ever count as a match (0.55 for ViT-B/16).
Peak	The mean of the 100 highest relevancies of a query; the threshold is set halfway between floor and peak.
Hit	The top candidate’s box (grown by 0.3 m) contains an annotated instance, or its centre is within 0.75 m of one.
L-C / L-CD	Lift table, queried with CLIP only / with DINO smoothing and splitting at query time.
F-C / F-CD	FMGS trained on CLIP alone / with its paper losses (CLIP + DINO + pixel alignment).
AUROC	Probability that a present object’s peak relevancy exceeds an absent object’s; 1 = perfect separation.

References

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis. *3D Gaussian Splatting for Real-Time Radiance Field Rendering*. ACM TOG 2023.
- [2] A. Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. ICML 2021.
- [3] M. Oquab et al. *DINOv2: Learning Robust Visual Features without Supervision*. TMLR 2024.
- [4] A. Kirillov et al. *Segment Anything*. ICCV 2023.
- [5] J. Kerr, C. M. Kim, K. Goldberg, A. Kanazawa, M. Tancik. *LERF: Language Embedded Radiance Fields*. ICCV 2023.
- [6] M. Qin, W. Li, J. Zhou, H. Wang, H. Pfister. *LangSplat: 3D Language Gaussian Splatting*. CVPR 2024.
- [7] X. Zuo, P. Samangouei, Y. Zhou, Y. Di, M. Li. *FMGS: Foundation Model Embedded 3D Gaussian Splatting for Holistic 3D Scene Understanding*. IJCV 2024.

- [8] T. Müller, A. Evans, C. Schied, A. Keller. *Instant Neural Graphics Primitives with a Multiresolution Hash Encoding*. ACM TOG 2022.
- [9] J. Cheng, J.-N. Zaeck, L. Van Gool, D. P. Paudel. *Occam’s LGS: An Efficient Approach for Language Gaussian Splatting*. 2024.
- [10] J. Marrie, R. Ménégaux, M. Arbel, D. Larlus, J. Mairal. *LUDVIG: Learning-free Uplifting of 2D Visual Features to Gaussian Splatting Scenes*. 2024.